

| Question                                                                  | Short Answer                                                                                                                                                |
|---------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1. What is the difference between a compiler and an interpreter?          | A <b>compiler</b> translates the entire program into machine code before execution, while an <b>interpreter</b> translates and executes one line at a time. |
| 2. What is the purpose of the main() function?                            | main() is the <b>entry point</b> of a C/C++ program where execution begins.                                                                                 |
| 3. Why do we include header files like stdio.h or iostream?               | Header files provide <b>predefined functions, classes, and declarations</b> needed by the program.                                                          |
| 4. What is the difference between gcc and g++?                            | gcc is primarily used to compile <b>C programs</b> , while g++ is used to compile <b>C++ programs</b> and automatically links the C++ standard library.     |
| 5. What is an executable file?                                            | An executable file (.exe on Windows) is the <b>compiled program</b> that the operating system can run.                                                      |
| 6. What happens during compilation?                                       | The compiler checks for errors and converts <b>source code into machine code (executable)</b> .                                                             |
| 7. What is the difference between compile-time and run-time errors?       | <b>Compile-time errors</b> occur before execution (syntax errors), while <b>run-time errors</b> occur during program execution (e.g., division by zero).    |
| 8. What is the purpose of the return 0; statement?                        | It indicates that the program <b>completed successfully</b> and returns control to the operating system.                                                    |
| 9. Why is C called a procedural programming language?                     | Because programs are organized into <b>functions (procedures)</b> that execute step by step.                                                                |
| 10. What are the advantages of using an IDE over a command-line compiler? | An IDE provides <b>code editing, debugging, auto-completion, project management, and one-click build/run</b> , making development easier.                   |

| Question                          | Short Answer                                                                                                               |
|-----------------------------------|----------------------------------------------------------------------------------------------------------------------------|
| 1. Is every program an algorithm? | <b>No.</b> A program is the implementation of an algorithm, but poorly written programs may not follow a proper algorithm. |

|                                                                                  |                                                                                                                          |
|----------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------|
| <b>2. Is every algorithm a program?</b>                                          | <b>No.</b> An algorithm is a logical solution and can exist without being coded.                                         |
| <b>3. Can an algorithm exist without a programming language?</b>                 | <b>Yes.</b> Algorithms can be written in plain English, flowcharts, or pseudocode.                                       |
| <b>4. Can there be more than one correct algorithm for the same problem?</b>     | <b>Yes.</b> Different algorithms can solve the same problem with different efficiencies.                                 |
| <b>5. Is pseudocode executable?</b>                                              | <b>No.</b> Pseudocode is only a human-readable representation of logic.                                                  |
| <b>6. Is a flowchart mandatory before coding?</b>                                | <b>No.</b> It is optional but helps in planning and understanding complex logic.                                         |
| <b>7. Can a program compile successfully but still produce incorrect output?</b> | <b>Yes.</b> Logical errors can produce wrong results even if the program compiles successfully.                          |
| <b>8. Does compilation guarantee that the program is correct?</b>                | <b>No.</b> Compilation only checks syntax and some semantic errors, not program logic.                                   |
| <b>9. Can an algorithm have infinite steps?</b>                                  | <b>No.</b> A valid algorithm must terminate after a finite number of steps.                                              |
| <b>10. Can a flowchart have multiple end points?</b>                             | <b>Yes.</b> Depending on the problem, a flowchart can have multiple valid exit points.                                   |
| <b>11. Why do computers understand machine language but not C?</b>               | <b>Because the CPU executes only binary machine instructions (0s and 1s).</b> C must first be translated by a compiler.  |
| <b>12. Why can't humans directly write machine code for large software?</b>      | <b>Because machine code is difficult to read, write, debug, and maintain.</b> High-level languages improve productivity. |
| <b>13. Which takes more memory: source code or executable?</b>                   | <b>Generally, the executable file</b> because it contains machine code and linked libraries (though this can vary).      |
| <b>14. Is faster execution always better?</b>                                    | <b>No.</b> A balance between speed, readability, maintainability, and memory usage is often more important.              |

**15. Which is more important:  
readability or optimization?**

**Readability first, optimization when needed.** Write correct and maintainable code before optimizing for performance.